CLOUD & DEVOPS

Docker Deep Dive Visual Guide

Layers, cache, multi-stage builds — fast, small, secure images.

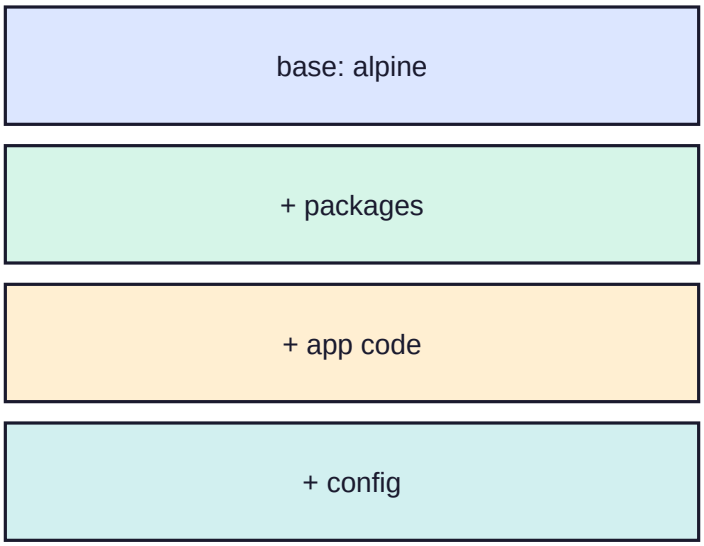
ANIMATED WALKTHROUGH

Step through pages to play the animation

STEP 1/6

Image Layers: Copy-on-Write Stack

Each Dockerfile instruction adds a read-only layer.



Container layer (writable, thin) sits on top — delete the container, the layer goes with it.

Layers are content-addressed and SHARED: ten images on one base share one copy.

STEP 2/6

Dockerfile Best Practices

Order matters: stable layers first, volatile last.

GOOD order

```
COPY package.json . && npm ci    # deps change rarely
COPY src/ ./                     # code changes often
```

FROM → deps → code → config

BAD order

Every code change invalidates the npm layer → 5-minute rebuilds, every time.

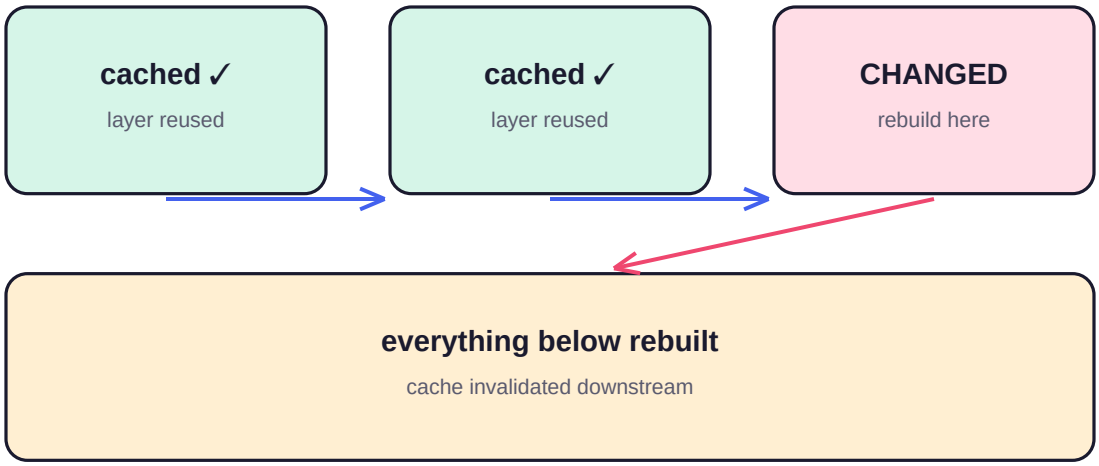
code copied BEFORE deps install

.dockerignore is mandatory: keep node_modules, .git, and secrets OUT of the build context.

STEP 3/6

Build Cache Mechanics

Docker reuses layers until the first changed instruction.

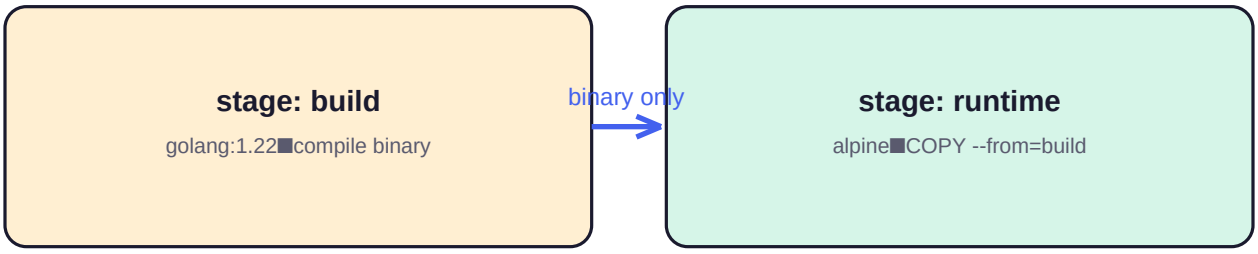


BuildKit (--progress=plain) shows exactly which layer broke the cache. Combine RUN steps to shrink layer count.

STEP 4/6

Multi-Stage Builds

Build with the heavy toolchain; ship only the artifact.



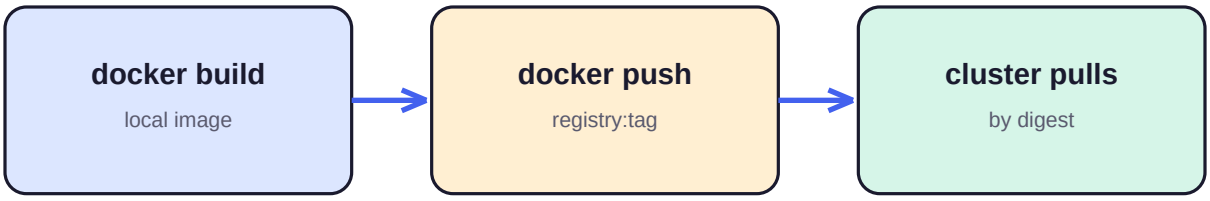
Result: 800MB build image → 25MB runtime image. No compiler, no source, smaller attack surface.

- `FROM golang AS build` then `FROM alpine` — final image starts from the last stage.
- `COPY --from=build /app/bin /usr/local/bin/app`: take only what you need.
- Use distroless or minimal bases for production — fewer CVEs to patch.

STEP 5/6

Registries: Push, Pull, Scan

The artifact store every deploy pulls from.



- Tag immutably: `app:1.4.2` and `app:1.4.2-<sha>` — never deploy `:latest` to prod.
- Scan on push: registries flag CVEs before the image can deploy.
- Pull by DIGEST in prod (`app@sha256:…`) — tags can be re-pointed, digests can't.

STEP 6/6

Run It Right: Runtime Hygiene

Small images still need safe runtime defaults.

Non-root USER

add `USER app` — a container breakout shouldn't mean root

Read-only FS

`--read-only` + tmpfs for scratch — malware can't persist

Resource limits

memory/CPU caps — one container can't starve the host

No secrets in ENV

mount secrets from a manager, not baked into the image

STEP ✓/✓

Key Takeaways

Docker Deep Dive — Remember

- Images are stacked read-only layers; containers add one thin writable layer.
- Order Dockerfile steps: stable (deps) first, volatile (code) last — cache wins.
- Multi-stage builds ship only the artifact: smaller, faster, fewer CVEs.
- Registries: immutable tags, scan on push, pull by digest in prod.
- Run as non-root, read-only FS, with resource limits — images are only half the story.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev